

1)

```
import java.io.*;
import java.util.*;
class product {
    private int id;
    private int price;
    private String name;
```

```
product () {
```

```
}
```

```
product (int id; String name; int price) {
```

```
    this.id = id;
```

```
    this.name = name;
```

```
    this.price = price;
```

```
}
```

```
public int getId() {
    return id;
```

```
}
```

```
public void setId (int id) {
```

```
    this.id = id;
```

```
}
```

```
public String getName() {
```

```
    return name;
```

```
}
```

```
public void setName (String name) {
```

```
    this.name = name;
```

```
public int price getPrice() {
```

```
    return price;
```

```
public void getPrice (int price) {
```

```
    this.price = price;
```

```
}
```

```
}
```

```

class productdemo {
    public static void main (String [] args) {
        product p = new product ();
        p.set Id (01);
        p.set Name ("mobile");
        p.set Price (30,000);
        System.out.println ("Id = " + p.getId ());
        System.out.println ("Name = " + p.getName ());
        System.out.println ("Price = " + p.getPrice ());
        product p2 = new product (02, "laptop", 50,000);
        System.out.println ("Id = " + p2.getId ());
        System.out.println ("Name = " + p2.getName ());
        System.out.println ("Price = " + p2.getPrice ());
    }
}

```

constructor overloading:

```

import java.io.*;
import java.util.*;

class Math {
    private int a, b, c;

    public Math () {
        int a = 100;
        int b = 50;
    }

    public Math (int xa, int yb) {
        a = x;
        b = y;
    }

    public void addition () {
        c = a + b;
        System.out.println ("addition result = " + c);
    }

    public void subtraction () {
        c = a - b;
        System.out.println ("subtraction result = " + c);
    }
}

```

```

public void multiplication() {
    c = a * b;
    System.out.println("multiplication result=" + c);
}

```

```

class constructoroverloadingdemo {
    public static void main(String[] args) {
        Math m1 = new Math(1);
        Math m2 = new Math(60, 20);
        m1.addition(); // 150
        m2.addition(); // 80
        m1.subtraction(); // 50
        m2.subtraction(); // 40
        m1.multiplication(); // 1500
        m2.multiplication(); // 1200
    }
}

```

3) Interface

Interface is collection of declaration

- * Interface support multiple inheritance
- * It does not allow access specifier
- * It allow only declarations.

It is like a blueprint.

```

import java.io.*;
import java.util.*;
interface iBank {

```

```

    void deposit();
    void withdraw();
}

```

```

interface print extends iBank {

```

```

    void printstatement();
}

```

class hdfc implements iBank, print {

Print void deposit () {

System.out.println("code to deposit");

Public void withdraw () {

System.out.println("code to withdraw");

Public void printStatement () {

System.out.println("code to print statement");

}

multiple inheritance

